

Trabalho 1

Análise de complexidade em estruturas de listas

Instituto Federal do Espírito Santo — Campus Serra
Técnicas de Programação Avançada
Prof. Victorio Albani de Carvalho

Integrantes do grupo

Lucas Ramos Pianissola
Ryan Lima Peçanha
Davi Alvarenga

Repositório do projeto

https://github.com/ryanlimap/trabalho_tpa

1. Desenvolvimento da biblioteca e do programa de teste

1.1 Atuação de cada componente do grupo

O quadro abaixo descreve a participação de cada integrante nos itens 1 (biblioteca de listas) e 2 (programa de teste) da especificação. O trabalho foi dividido por módulo, mas as decisões de projeto e a revisão do código foram discutidas pelos três.

Integrante	Atuação nos itens 1 e 2
Lucas Ramos Pianissola	Item 1 — a biblioteca: classes <code>No</code> e <code>ListaEncadeada</code> , com <i>Generics</i> e <code>Comparator</code> ; inserção ordenada e não-ordenada, pesquisa, remoção, <code>toString</code> e <code>iterator</code> ; implementação da interface <code>IColecao</code> e testes dos casos de borda.
Ryan Lima Peçanha	Item 2 — o programa de teste: classe de domínio <code>Contato</code> , os dois comparadores e a agenda (<code>app.Programa</code>) com o menu completo, a leitura do arquivo de entrada e a medição dos tempos exibidos ao usuário.
Davi Alvarenga	Apoio ao item 2 e infraestrutura das medições: <code>app.GeradorArquivo</code> , <code>app.Benchmark</code> e <code>app.ContagemOperacoes</code> ; execução das rodadas, organização dos dados, gráficos e redação do relatório.

1.2 Como as ferramentas de IA foram utilizadas

Usamos uma ferramenta de IA generativa (Claude, da Anthropic) como apoio em três frentes bem delimitadas, descritas a seguir.

a) Aproveitamento do material da disciplina. Fornecemos à ferramenta os slides de revisão (“Lista encadeada e Generics”) e o repositório do professor, de onde ela extraiu os métodos já apresentados em aula — `inserirElemento`, `contemElemento`, `excluirElemento`, a inserção ordenada com `Comparator` e a interface `IColecao` — e os organizou como ponto de partida do nosso código.

b) Orientação sobre como usar esses métodos. A ferramenta explicou como adaptar o código dos slides às exigências do trabalho: usar *Generics* (T) no lugar de `Object`, receber o `Comparator<T>` e o boolean `ordenada` no construtor, adequar as assinaturas aos métodos exigidos pela `IColecao` (`adicionar`, `pesquisar`, `remover` e `quantidadeNos`) e tratar os casos de borda (lista vazia, remoção do primeiro e do último nó). Revisamos o resultado linha a linha e testamos cada caso antes de aceitá-lo.

c) Gráficos, tabelas e imagens do relatório. A ferramenta auxiliou na geração dos gráficos e das tabelas apresentados na seção 3, a partir do arquivo CSV produzido pelas nossas execuções: escrevemos com o apoio dela os scripts `analise/gerar_graficos.py` (matplotlib) e `analise/gerar_relatorio.py` (ReportLab), que leem as medições e montam as figuras e as tabelas. Todos os números apresentados são os medidos nas nossas próprias execuções — nenhum valor foi estimado ou inventado pela ferramenta.

d) Revisão de texto. A redação final do relatório foi revisada com apoio da ferramenta, para clareza e correção da linguagem.

1.3 Repositório e organização do código

Todo o código está disponível em https://github.com/ryanlimap/trabalho_tpa, com um arquivo `README.md` que descreve a organização das pastas e como compilar e executar o projeto.

Pasta / arquivo	Conteúdo
src/colecao/IColecao.java	Interface exigida pela especificação.
src/colecao/No.java	Nó genérico: valor do tipo T e referência para o próximo.
src/colecao/ListaEncadeada.java	A biblioteca: lista genérica, ordenada ou não, com Comparator. Não imprime nada.
src/app/Contato.java	Classe de domínio (nome e telefone); toString "nome-telefone".
src/app/ComparadorContatoPorNome.java	Comparator pela chave nome.
src/app/ComparadorContatoPorTelefone.java	Comparator pela chave telefone.
src/app/Programa.java	Agenda de contatos com o menu e a medição de tempos.
src/app/GeradorArquivo.java	Gera arquivos de entrada de qualquer tamanho.
src/app/Benchmark.java	Executa uma rodada da análise empírica (tempos) e imprime uma linha CSV.
src/app/ComparadorContador.java	Decorator que conta as chamadas ao comparador.
src/app/ContagemOperacoes.java	Conta o número de comparações executadas, em vez de medir tempo.
analise/	Scripts Python que produzem os gráficos e este PDF.
dados/	Arquivos de entrada e o CSV com todas as medições.

Decisão de projeto importante: o programa mantém **duas listas** — uma indexada por nome e outra por telefone — que guardam **as mesmas instâncias** de Contato. Não há duplicação de contatos: as duas listas apenas apontam para os mesmos objetos, de modo que uma alteração feita por um caminho é vista pelo outro.

2. Análise matemática de complexidade

Nesta seção analisamos os métodos adicionar, pesquisar, remover e quantidadeNos da classe ListaEncadeada. Em todos os trechos, a numeração das linhas é a numeração real do arquivo src/colecao/ListaEncadeada.java.

Premissas. (i) n é a quantidade de nós da lista; (ii) atribuições, comparações, criação de um nó e desvios são operações de custo constante, contadas como 1; (iii) a chamada `comparador.compare(...)` é considerada $O(1)$ — para as chaves usadas (nome e telefone) ela compara Strings de tamanho limitado, independente de n ; (iv) analisamos o **pior caso**, conforme discutido em aula, por ser o limite superior garantido para qualquer entrada.

2.1 Método adicionar

Listagem 1 — adicionar (ponto de entrada) — src/colecao/ListaEncadeada.java, linhas 46-58

```
46     @Override
47     public boolean adicionar(T novoValor) {
48         if (novoValor == null) {
49             return false;
50         }
51         if (this.ordenada) {
52             adicionarOrdenado(novoValor);
53         } else {
54             adicionarNaoOrdenado(novoValor);
55         }
56         this.quant++;
57         return true;
58     }
```

As linhas 48-50 fazem uma comparação e, no máximo, um retorno: custo 2. A linha 51 testa o atributo ordenada (custo 1) e desvia para um dos dois métodos auxiliares. As linhas 56-57 executam um incremento e um retorno: custo 2. Assim, $T_{\text{adicionar}}(n) = 5 + T_{\text{auxiliar}}(n)$: o custo do método é dominado pelo auxiliar escolhido.

Listagem 2 — inserção em lista NÃO-ORDENADA — src/colecao/ListaEncadeada.java, linhas 63-72

```
63     private void adicionarNaoOrdenado(T novoValor) {
64         No<T> novo = new No<>(novoValor);
65         if (this.prim == null) {
66             this.prim = novo;
67             this.ult = novo;
68         } else {
69             this.ult.setProx(novo);
70             this.ult = novo;
71         }
72     }
```

Não há laço algum. A linha 64 cria o nó (custo 1); a linha 65 testa se a lista está vazia (custo 1); em seguida executa-se o bloco 66-67 (duas atribuições) ou o bloco 69-70 (duas atribuições). O número de operações é o mesmo para uma lista vazia ou com um milhão de nós, porque a referência ult dá acesso direto ao fim da lista — não é preciso percorrê-la. Portanto $T(n) = 4$, uma constante, e a ordem de complexidade é **$O(1)$** , tanto no melhor quanto no pior caso.

Observação: se a classe não guardasse a referência ult, seria necessário percorrer a lista inteira para chegar ao último nó e a inserção passaria a ser $O(n)$.

Listagem 3 — inserção em lista ORDENADA — src/colecao/ListaEncadeada.java, linhas 78-101

```
78     private void adicionarOrdenado(T novoValor) {
79         No<T> novo = new No<>(novoValor);
80         No<T> atual = this.prim;
81         No<T> ant = null;
82         if (this.prim == null) {
83             this.prim = novo;
84             this.ult = novo;
85             return;
86         }
87         while (atual != null && comparador.compare(atual.getValor(), novoValor) < 0) {
88             ant = atual;
89             atual = atual.getProx();
90         }
91         if (ant == null) {
92             novo.setProx(this.prim);
93             this.prim = novo;
94         } else if (atual == null) {
95             this.ult.setProx(novo);
96             this.ult = novo;
97         } else {
98             ant.setProx(novo);
99             novo.setProx(atual);
100         }
101     }
```

As linhas 79-86 têm custo constante (criação do nó, duas atribuições, o teste de lista vazia e, quando ela está vazia, a saída imediata pela linha 85). O que define a complexidade é o laço das linhas 87-90: ele avança nó a nó enquanto o elemento atual for **menor** que o novo elemento. Cada iteração custa 3 operações (a comparação da linha 87 e as duas atribuições das linhas 88-89).

Seja k o número de nós visitados antes de encontrar a posição de inserção. O laço executa k iterações e, depois dele, as linhas 91-100 fazem no máximo dois testes e duas atribuições (custo constante 4). Logo $T(n) = 3k + c$.

Pior caso: ocorre quando o elemento a ser inserido é **maior que todos os que já estão na lista**, ou seja, quando ele deve ir para o fim. Nesse caso o laço percorre todos os n nós ($k = n$) e o custo é $T(n) = 3n + c$, ou seja, $O(n)$. É exatamente o que acontece quando o arquivo de entrada já está ordenado de forma crescente: cada novo contato é o maior de todos. **Melhor caso:** quando o novo elemento é menor que o primeiro ($k = 0$), o custo é constante, $O(1)$. **Caso médio** (entrada aleatória, como nos nossos testes): $k \approx n/2$, o que ainda é $O(n)$.

2.1.1 Consequência: custo de montar uma lista com n elementos

O programa de teste insere os n contatos do arquivo um a um. Na lista **não-ordenada**, cada inserção é $O(1)$, então a montagem completa custa $n \cdot O(1) = O(n)$. Na lista **ordenada**, a i -ésima inserção percorre em média $i/2$ nós e, no pior caso, i nós; somando para $i = 1 \dots n$:

$$1 + 2 + 3 + \dots + n = n(n+1)/2 = n^2/2 + n/2$$

Descartando o termo de menor grau e a constante multiplicativa, a montagem da lista ordenada é $O(n^2)$. Essa é a diferença mais importante entre as duas listas e é claramente visível nas medições da seção 3.

2.2 Método pesquisar

Listagem 4 — pesquisar — src/colecao/ListaEncadeada.java, linhas 107-121

```
107     @Override
108     public T pesquisar(T valor) {
109         No<T> aux = this.prim;
110         while (aux != null) {
111             int cmp = comparador.compare(aux.getValor(), valor);
112             if (cmp == 0) {
113                 return aux.getValor();
114             }
115             if (this.ordenada && cmp > 0) {
116                 return null;
117             }
118             aux = aux.getProx();
119         }
120         return null;
121     }
```

A linha 109 é uma atribuição (custo 1). O laço das linhas 110-119 percorre a lista nó a nó; cada iteração custa 5 operações (o teste da linha 110, a comparação da linha 111, os testes das linhas 112 e 115 e a atribuição da linha 118). Se o elemento é encontrado, a linha 113 o devolve; se a lista acaba, a linha 120 devolve `null`.

A novidade em relação ao código visto em aula é a linha 115: **em lista ordenada, assim que encontramos um elemento maior que o procurado, sabemos que o elemento buscado não existe** e interrompemos a busca. Isso evita percorrer o resto da lista, mas **não muda a ordem de complexidade**.

Pior caso — lista não-ordenada: o elemento procurado está no último nó ou não existe na lista; o laço executa n iterações e $T(n) = 5n + c$, ou seja, **$O(n)$** . É justamente o caso medido na seção 3, em que pesquisamos o **último contato do arquivo** — que, na lista não-ordenada, está no fim.

Pior caso — lista ordenada: ocorre quando o elemento procurado é o maior de todos (ou é maior que todos, no caso de uma busca sem sucesso). O laço também percorre os n nós e a complexidade continua **$O(n)$** . A parada antecipada só melhora o caso médio: em uma busca sem sucesso, ela cai de n para cerca de $n/2$ comparações — um ganho de fator constante, que a notação O despreza.

2.3 Método remover

Listagem 5 — remover — src/colecao/ListaEncadeada.java, linhas 127-152

```
127     @Override
128     public boolean remover(T valor) {
129         No<T> aux = this.prim;
130         No<T> ant = null;
131         while (aux != null) {
132             int cmp = comparador.compare(aux.getValor(), valor);
133             if (cmp == 0) {
134                 if (ant == null) {
135                     this.prim = aux.getProx();
136                 } else {
137                     ant.setProx(aux.getProx());
138                 }
139                 if (aux == this.ult) {
140                     this.ult = ant;
141                 }
142                 this.quant--;
143                 return true;
144             }
145             if (this.ordenada && cmp > 0) {
146                 return false;
147             }
148             ant = aux;
149             aux = aux.getProx();
150         }
151         return false;
152     }
```

A estrutura é a mesma da pesquisa: as linhas 129-130 custam 2 e o laço das linhas 131-150 percorre a lista, com custo constante por iteração (o teste da linha 131, a comparação da linha 132 e as atribuições das linhas 148-149). Quando o elemento é encontrado, as linhas 133-143 fazem apenas **ligações de ponteiros**: tratam o caso de ser o primeiro nó (linhas 134-135), o caso geral (linha 137), o caso de ser o último nó (linhas 139-141, que atualizam `ult`), decrementam o contador e retornam. Todo esse bloco é $O(1)$ — **a remoção em si é barata; caro é encontrar o nó**.

Pior caso: o elemento está no último nó ou não está na lista, e o laço executa n iterações: **$O(n)$** . Vale para os dois tipos de lista. Na lista ordenada a linha 145 novamente interrompe a busca mais cedo quando o elemento não existe, o que não altera a ordem de complexidade.

Note que a remoção do último nó, embora seja $O(1)$ na parte de religação, exige percorrer a lista inteira para encontrar o nó **anterior** a ele — consequência de a lista ser **simplesmente** encadeada. Com uma lista duplamente encadeada esse passo seria $O(1)$, mas a busca pelo nó continuaria $O(n)$.

2.4 Método quantidadeNos

Listagem 6 — quantidadeNos — src/colecao/ListaEncadeada.java, linhas 158-161

```
158     @Override
159     public int quantidadeNos() {
160         return this.quant;
161     }
```

O método executa uma única instrução: devolve o atributo `quant`. Como esse contador é mantido atualizado em `adicionar` (linha 56) e em `remover` (linha 142), não é necessário percorrer a estrutura. Portanto **$T(n) = 1$** e a complexidade é **$O(1)$** — no melhor e no pior caso, em lista ordenada ou não. Se o contador não existisse, seria preciso percorrer a lista contando os nós, e o método seria $O(n)$; o custo dessa escolha é apenas um incremento por inserção, já contabilizado na análise do `adicionar`.

2.5 Resumo e comparação entre lista ordenada e não-ordenada

Método	Não-ordenada (pior caso)	Ordenada (pior caso)	Quando o pior caso ocorre
adicionar	$O(1)$	$O(n)$	Ordenada: novo elemento maior que todos (entrada já ordenada de forma crescente). Não-ordenada: não há pior caso — é sempre constante.
pesquisar	$O(n)$	$O(n)$	Elemento no último nó ou inexistente (na ordenada, maior que todos).
remover	$O(n)$	$O(n)$	Elemento no último nó ou inexistente (na ordenada, maior que todos).
quantidadeNos	$O(1)$	$O(1)$	Não existe pior caso: custo fixo.
montar n elementos	$O(n)$	$O(n^2)$	Consequência direta do custo de cada inserção (seção 2.1.1).

Há diferença de ordem de complexidade entre os dois tipos de lista? Sim, mas apenas na **inserção**: $O(1)$ na não-ordenada contra $O(n)$ na ordenada (e, consequentemente, $O(n)$ contra $O(n^2)$ para montar a estrutura inteira). Para pesquisar, remover e `quantidadeNos` **a ordem de complexidade é a mesma**.

Isso pode parecer contra-intuitivo: ordenar normalmente serve para acelerar a busca. O ganho não aparece porque a lista encadeada **não permite acesso direto a uma posição qualquer** — para chegar ao nó do meio é preciso passar por todos os anteriores, o que inviabiliza a busca binária. A ordenação só permite **interromper antes** uma busca sem sucesso, reduzindo o número médio de comparações pela metade: um ganho de fator constante, desprezado pela notação O . Em resumo, **pelo nosso código a lista ordenada paga um preço alto na montagem ($O(n^2)$) sem obter melhora na ordem de complexidade das consultas**; ela se justifica quando é necessário percorrer os dados em ordem, não por desempenho de busca.

3. Análise empírica de complexidade

3.1 Metodologia

Para cada tamanho de entrada geramos um arquivo com contatos aleatórios usando a classe `app.GeradorArquivo` (nomes formados por primeiro nome, sobrenome e um número aleatório; telefones únicos, obtidos pelo embaralhamento de uma faixa de números). Em seguida, a classe `app.Benchmark` executa, para cada arquivo e para cada tipo de lista, os quatro passos pedidos na especificação:

1) ler o arquivo e montar as duas listas (por nome e por telefone), medindo o tempo total; 2) pesquisar pelo **telefone do último contato do arquivo**; 3) pesquisar pelo **nome do último contato do arquivo**; 4) remover o **último contato do arquivo**, pelo telefone.

Os tempos são medidos com `System.nanoTime()` e convertidos para milissegundos. Cada medição roda em uma **JVM nova**, para que uma execução não interfira na seguinte. As execuções foram repetidas 3 vezes e reportamos a **mediana**, que é menos sensível a variações pontuais do sistema operacional e do coletor de lixo. As duas exceções são a lista **ordenada** em 100.000 e em 200.000 contatos: como cada uma dessas execuções leva minutos (a de 200.000 levou 20,1 minutos), elas foram rodadas **uma única vez** — nesses dois pontos o valor da tabela é uma medição isolada, não uma mediana.

Uma observação de implementação: o `System.out.printf` das classes de medição usa `Locale.US`. Sem isso, em um Windows configurado em português o Java imprimiria os tempos com vírgula decimal (32,4520) e o CSV não seria lido corretamente pelos scripts de análise.

Ambiente de execução: AMD Ryzen 5 5600G (6 núcleos, 16 MB de cache L3), 16 GB de RAM DDR4 2666 MHz, Windows 11, OpenJDK 21 (64 bits). Todas as medições apresentadas nesta seção foram coletadas nessa mesma máquina, sem outros programas concorrendo pela CPU.

Tamanhos usados: 25.000, 50.000, 100.000, 200.000, 400.000 contatos para a lista não-ordenada e 25.000, 50.000, 100.000, 200.000 para a ordenada. Não incluímos a lista ordenada no tamanho de 400.000 porque, sendo a montagem $O(n^2)$, a execução levaria várias horas — o que, por si só, já é uma confirmação prática da análise matemática.

Medição complementar. Além do tempo de relógio, instrumentamos o código para **contar o número de comparações** executadas (classes `app.ComparadorContador` e `app.ContagemOperacoes`). O tempo depende da máquina, do JIT e dos caches do processador; a contagem de operações, não. Ela mede exatamente aquilo que a análise matemática da seção 2 prevê, e por isso é a evidência mais limpa da ordem de crescimento.

3.2 Tempo de leitura do arquivo e montagem das listas

n	Montagem — não-ordenada (ms)	razão	Montagem — ordenada (ms)	razão
25.000	34,31	—	5.248,54	—
50.000	42,04	1,23×	57.767,49	11,01×
100.000	73,64	1,75×	261.409,72	4,53×
200.000	102,03	1,39×	1.204.511,54	4,61×
400.000	168,48	1,65×	não medido	—

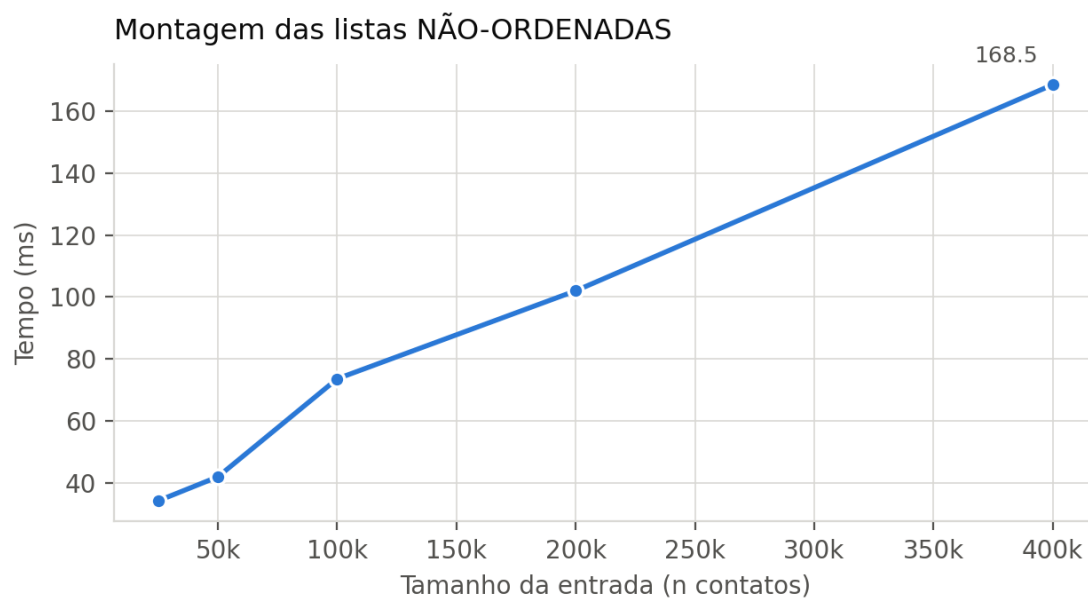


Figura 1 — Montagem das listas não-ordenadas (mediana, em ms).

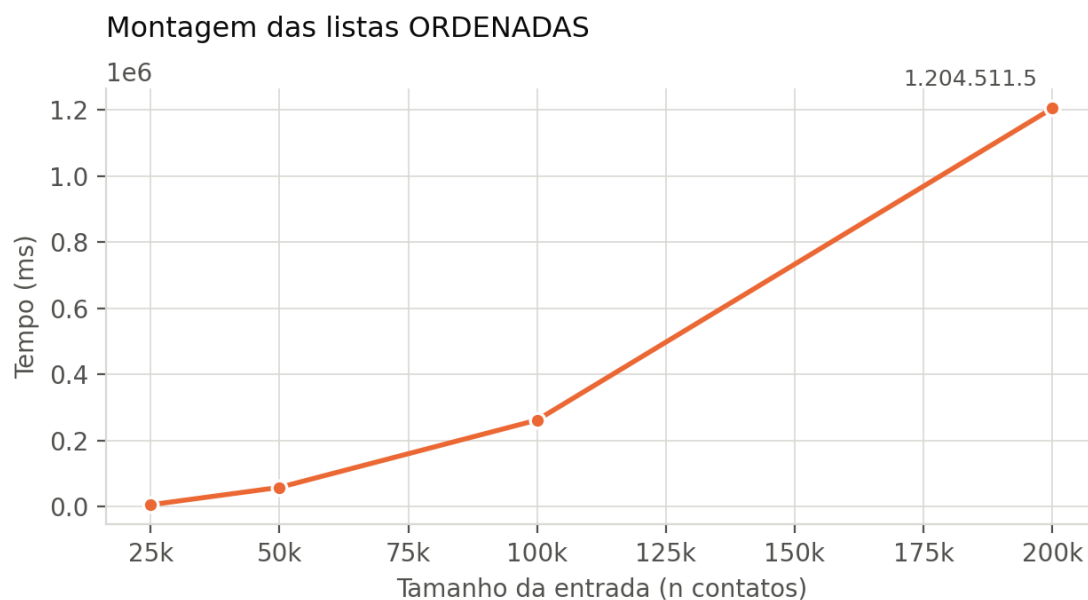


Figura 2 — Montagem das listas ordenadas (mediana, em ms). Note a escala: segundos, não milissegundos.

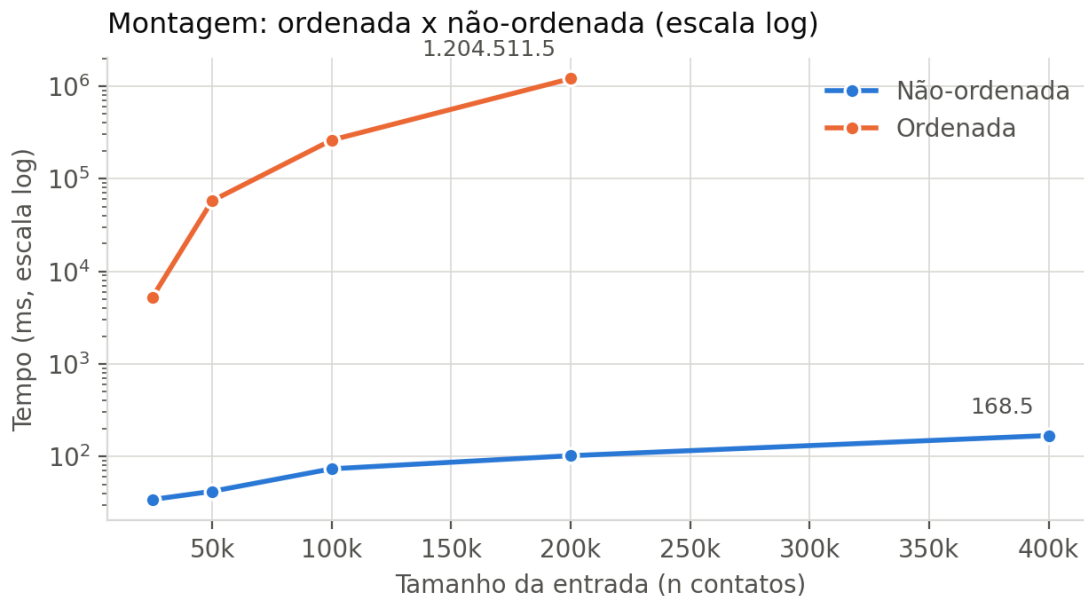


Figura 3 — As duas curvas na mesma escala logarítmica, para comparação.

Interpretação. As colunas “razão” das tabelas mostram por quanto o tempo é multiplicado quando n dobra — é a forma mais direta de identificar a ordem de crescimento: em uma função linear a razão tende a 2; em uma função quadrática, a 4.

Na lista **não-ordenada** todas as razões ficam **abaixo de 2** (entre 1,2 e 1,8), e no conjunto o tempo cresce cerca de 4,9 vezes enquanto n cresce 16 vezes. O crescimento é, portanto, **no máximo linear** — e está longe de qualquer coisa quadrática. As razões menores que 2 se explicam pelos **custos fixos** embutidos na medição: inicialização da JVM, compilação JIT dos métodos e leitura do arquivo do disco não dependem de n e pesam proporcionalmente muito mais nas entradas pequenas. A prova limpa de que cada inserção é $O(1)$ está na seção 3.2.1: a montagem não-ordenada executa **zero** comparações, em qualquer tamanho.

Na lista **ordenada** as duas últimas razões são **4,53** e **4,61** — a assinatura do comportamento **quadrático $O(n^2)$** previsto na seção 2.1.1. Já a primeira duplicação (25.000 → 50.000) rendeu uma razão de **11,0**, bem acima de 4. Esse ponto fora da curva **não** significa que o algoritmo seja pior que quadrático, e a contagem de operações é categórica quanto a isso: entre esses mesmos dois tamanhos o número de comparações cresceu **3,99x**, e entre 50.000 e 100.000, **4,00x**. O que muda é o **custo de cada** comparação, como explicamos logo a seguir.

Com 200.000 contatos, montar as listas ordenadas levou **20,1 minutos**, contra **102,0 ms** das não-ordenadas — cerca de **11.806 vezes mais**. E essa diferença não é constante: ela **aumenta** com n , porque n^2 cresce mais rápido que n . É a diferença entre $O(n)$ e $O(n^2)$ aparecendo no relógio.

3.2.1 Contagem de operações: a evidência mais limpa

A tabela abaixo mostra quantas **comparações entre contatos** foram efetivamente executadas para montar as duas listas (soma das duas) e para pesquisar o telefone do último contato do arquivo.

n	Montagem não-ordenada	Montagem ordenada	razão	Pesquisa não-ordenada
25.000	0	313.072.369	—	25.000
50.000	0	1.248.094.603	3,99×	50.000
100.000	0	4.986.760.666	4,00×	100.000
200.000	0	não medido	—	200.000
400.000	0	não medido	—	400.000

Os números são categóricos:

- na lista **não-ordenada** a montagem faz **zero comparações** — a inserção no fim, apoiada na referência `ult`, não precisa comparar nada, confirmando o $O(1)$ por inserção deduzido na Listagem 2;
- na lista **ordenada**, o número de comparações é multiplicado por **3,99×** e **4,00×** a cada vez que n dobra — a assinatura de $\Theta(n^2)$, exatamente como previsto pela soma $1 + 2 + \dots + n = n^2/2 + n/2$;
- a pesquisa na lista não-ordenada faz **exatamente n comparações** (25.000 comparações para $n = 25.000$, e assim por diante): é o pior caso de $O(n)$ acontecendo literalmente, como analisado na seção 2.2.

A tabela não traz a contagem da pesquisa na **lista ordenada**, e a razão é metodológica. Nela a busca para assim que encontra um elemento maior que o procurado (linha 115 da Listagem 4), de modo que o número de comparações é a **posição** do telefone procurado na ordem crescente — um valor sorteado pelo gerador, que não depende do tamanho da entrada. Nas nossas execuções essa contagem deu 14.237 tanto para 25.000 quanto para 50.000 contatos, e 64.237 para 100.000: com a mesma semente, o gerador sorteia posições congruentes entre si. Esses valores medem o sorteio, não o crescimento, e por isso não podem ser lidos como uma curva — o que eles ilustram bem é o efeito da parada antecipada, discutido na seção 2.2.

Por que o tempo saltou 11× entre 25.000 e 50.000, se as operações só quadruplicaram? A diferença não está no algoritmo, e sim na **hierarquia de memória**. A inserção ordenada percorre a lista seguindo referências de um nó para o outro, e esses nós estão espalhados pela memória. Com 25.000 contatos, boa parte das duas listas ainda cabe nos **16 MB de cache L3** do Ryzen 5 5600G e cada passo custa poucos ciclos; com 50.000 a estrutura já não cabe, e quase todo passo passa a ser um acesso à memória principal, dezenas de vezes mais lento. A partir daí o custo por operação se estabiliza — as duas razões seguintes voltam para perto de 4,5. Ou seja, o que aumentou nesse trecho foi o **custo médio de cada operação**, uma constante multiplicativa que a notação O , por definição, despreza. A ordem de complexidade continua $O(n^2)$; o relógio apenas mostra que, na prática, as constantes também importam.

Esse é justamente o motivo de termos medido as duas coisas. A **contagem de operações** é o argumento correto para a ordem de complexidade (e ela confirma o quadrático com precisão de duas casas); o **tempo** é o argumento para o custo de memória. Usar só o relógio levaria à conclusão errada de que o algoritmo piora em 25.000 e melhora depois.

3.3 Tempo de pesquisa

Pesquisa pelo **telefone** do último contato do arquivo:

n	Pesquisa por telefone — não-ordenada (ms)	razão	Pesquisa por telefone — ordenada (ms)	razão
25.000	5,00	—	0,99	—
50.000	6,43	1,29×	1,83	1,86×
100.000	8,63	1,34×	10,47	5,73×
200.000	12,90	1,49×	19,81	1,89×
400.000	14,67	1,14×	não medido	—

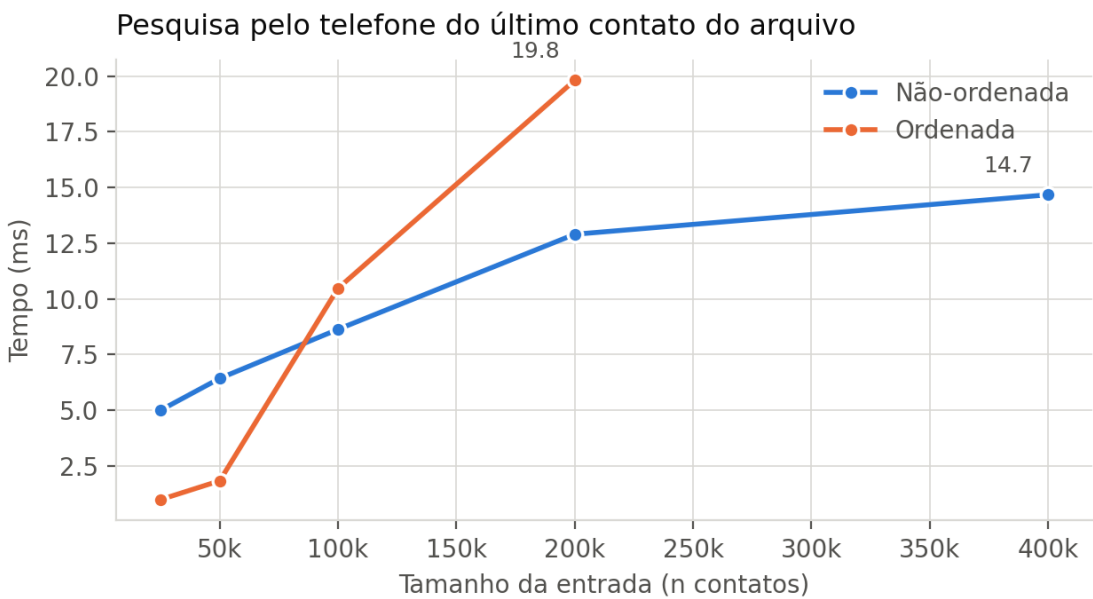


Figura 4 — Pesquisa pelo telefone do último contato do arquivo.

Pesquisa pelo **nome** do último contato do arquivo:

n	Pesquisa por nome — não-ordenada (ms)	razão	Pesquisa por nome — ordenada (ms)	razão
25.000	2,69	—	0,69	—
50.000	2,02	0,75×	3,77	5,43×
100.000	7,04	3,49×	5,00	1,33×
200.000	4,74	0,67×	13,22	2,64×
400.000	6,37	1,34×	não medido	—

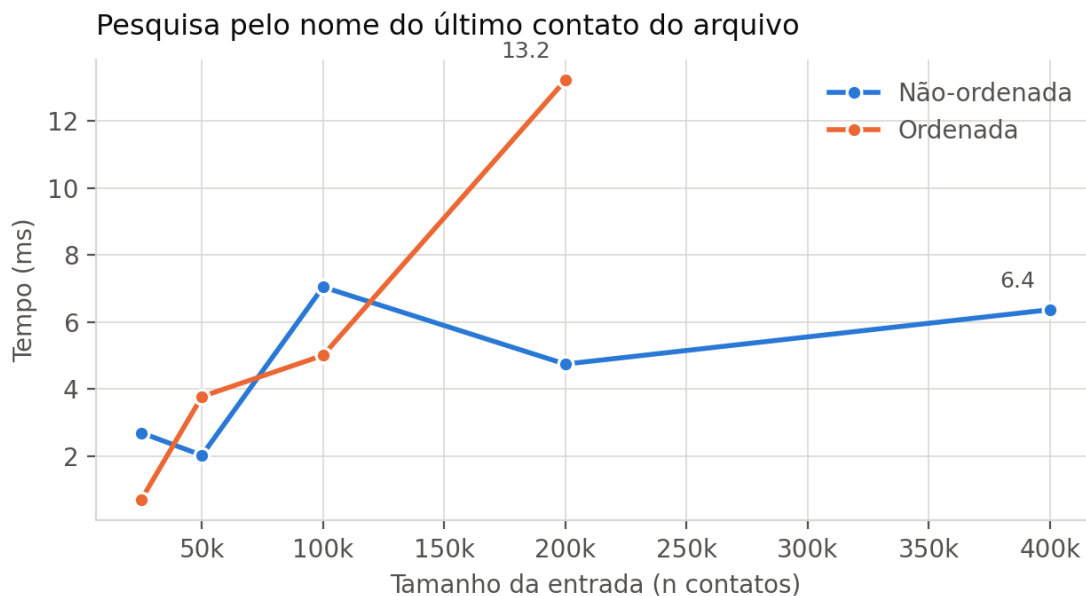


Figura 5 — Pesquisa pelo nome do último contato do arquivo.

Interpretação. Na lista **não-ordenada**, o contato procurado é o último do arquivo e, portanto, está no **último nó** da lista: é o **pior caso** descrito na seção 2.2. O tempo cresce com n , confirmando o comportamento $O(n)$.

Na lista **ordenada** os tempos são **menores** nos dois primeiros tamanhos, e isso merece explicação — afinal, a análise matemática diz que ambas são $O(n)$. Há dois motivos, e nenhum deles contradiz a teoria. Primeiro, como os dados de entrada são aleatórios, o último contato do arquivo ocupa uma **posição qualquer** da lista ordenada (em média o meio, $n/2$ nós), enquanto na não-ordenada ele está garantidamente no fim (n nós): é a diferença entre o caso médio e o pior caso, um fator constante de 2. Segundo, na execução ordenada a montagem demorou segundos, tempo mais que suficiente para a JVM compilar os métodos da lista para código nativo, o que torna cada iteração mais barata. Ambos os efeitos são **constantes multiplicativas**, justamente o que a notação O despreza.

A partir de 100.000 contatos a relação se inverte e a busca na lista ordenada passa a ser a mais lenta. É o mesmo fenômeno descrito na seção 3.2.1: nessa execução a lista ordenada foi construída por inserções que espalharam os nós pela memória, de modo que percorrê-la gera muito mais faltas de cache. De novo, trata-se do **custo de cada passo**, não da quantidade de passos — a contagem de operações confirma que a pesquisa continua percorrendo $O(n)$ nós nos dois casos.

As medições de busca são as mais ruidosas do trabalho, porque duram poucos milissegundos e sofrem interferência do sistema operacional, do coletor de lixo e do estado dos caches do processador. Por isso a razão entre medidas consecutivas oscila bastante — de 0,67 a 5,43 nas nossas tabelas — em vez de se aproximar de 2. A tendência geral de crescimento linear, porém, é clara, e a contagem de operações (exatamente n comparações na lista não-ordenada) não deixa dúvida sobre a ordem de complexidade.

3.4 Tempo de remoção

n	Remoção — não-ordenada (ms)	razão	Remoção — ordenada (ms)	razão
25.000	2,25	—	1,14	—

n	Remoção — não-ordenada (ms)	razão	Remoção — ordenada (ms)	razão
50.000	3,67	1,63×	2,26	1,98×
100.000	6,59	1,80×	10,15	4,49×
200.000	8,46	1,28×	20,08	1,98×
400.000	11,07	1,31×	não medido	—

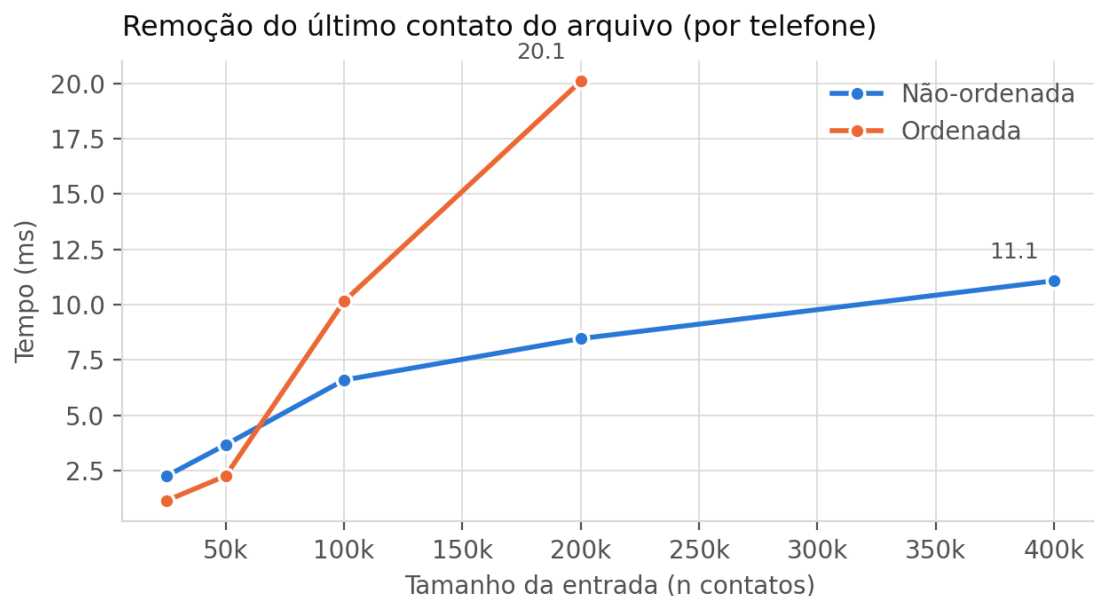


Figura 6 — Remoção do último contato do arquivo, pesquisado pelo telefone.

Interpretação. Os tempos de remoção acompanham de perto os de pesquisa, como previsto na seção 2.3: remover custa **encontrar o nó** ($O(n)$) mais um número fixo de religações de ponteiros ($O(1)$). Na lista não-ordenada, novamente, o elemento removido é o último nó — o pior caso — e o crescimento é linear com n .

3.5 Conclusão

Os dados coletados **atendem ao esperado** pela análise matemática da seção 2:

- a montagem da lista **não-ordenada** cresce de forma **no máximo linear** (todas as razões abaixo de 2; 4,9× de tempo para 16× de entrada), como previsto por $O(n)$, porque cada inserção é $O(1)$ graças à referência `ult` — e, de fato, a montagem executa zero comparações;
- a montagem da lista **ordenada** cresce quadraticamente — a contagem de comparações é multiplicada por 3,99× e 4,00× a cada duplicação de n —, como previsto por $O(n^2)$, porque cada inserção precisa percorrer a lista até achar a posição correta;
- **pesquisa e remoção** crescem linearmente nos dois tipos de lista, como previsto por $O(n)$; a ordenação altera o tempo apenas por **fatores constantes** — para melhor nas entradas menores (parada antecipada e posição média do elemento) e para pior nas maiores (faltas de cache) —, mas **não muda a ordem de complexidade**;
- quantidade de nós é $O(1)$ e seu tempo é indistinguível de zero em qualquer tamanho de entrada, por isso não foi incluído nos gráficos.

A principal lição prática é que, em uma lista **simplesmente encadeada**, manter os elementos ordenados custa caro ($O(n^2)$ para montar) e não traz o ganho que se poderia esperar nas consultas,

pois a estrutura não permite acesso direto a uma posição e, portanto, não permite busca binária. Estruturas que oferecem esse acesso — vetores ordenados, árvores de busca balanceadas ou tabelas hash — são as alternativas indicadas quando a busca precisa ser mais rápida que $O(n)$.

Por fim, os desvios observados — razões abaixo de 2 nas entradas pequenas da lista não-ordenada e o salto de $11\times$ entre 25.000 e 50.000 na ordenada — reforçam um ponto discutido em aula: a análise assintótica descreve a **tendência** do algoritmo quando n cresce, contando operações. O tempo de relógio embute constantes que a notação O despreza (inicialização da JVM, compilação JIT, acesso ao disco e, principalmente, o custo de acesso à memória quando a estrutura deixa de caber no cache). Por isso medimos as duas coisas: elas respondem a perguntas diferentes e, juntas, contam a história completa.